Week 3 Lecture Notes: Abstract Classes, Dataclasses, Typing and a Composable `DataPipeline`

One reproducible recipe for cleaning sensor data

AI for Aerospace & Mechanical Engineering (PhD), Block 0

> **Provenance:** new content. The instructor's PDFs do not cover Python OOP. The Python material is rewritten (not copied) from W3Schools (Python abc Module, dataclasses Module, typing Module, `__repr__()` and `__eq__()` methods, Iterators) and GeeksforGeeks (Abstract Classes in Python, Understanding Python Dataclasses, Python Generics, str() vs repr()). The Hampel filter is adapted from R. K. Pearson, "Outliers in process modeling and identification", *IEEE Trans. Control Systems Technology* 10(1), 2002; the MAD consistency constant from P. J. Rousseeuw and C. Croux, *JASA* 88(424), 1993.

1. The problem

The structures group asks for the peak load factor of the Week 1 test flight. Analyst A smooths the accelerometer with a 5-sample moving average and reports **1.87 g at t = 66.7 s**. Analyst B first takes a 7-sample rolling median, then the same average, and reports **1.80 g at t = 192 s**. Both scripts are short and reasonable, and both ran without error.

B is right: 192 s is the pull-up manoeuvre, while 66.7 s is level cruise with one corrupted sample. The engineering lesson (despike before smoothing) is the easy part. The software lesson is that nothing in either result says how it was produced: the **order** of the steps lives inside a function body, the **parameters** are not stored with the number, two scripts cannot be **compared**, and a step that receives the **wrong kind of data** fails late or not at all. This week we turn a cleaning recipe into an *object*: an ordered chain of typed stages that prints itself, compares by value, checks its own wiring, and saves itself as JSON.

Worked Example 1: one spike through a moving average

In cruise the vertical specific force is $a_z \approx 10.35$ m/s² ($n \approx 1.05$). One sample reads 50.33 m/s², a single-sample glitch of about +40 m/s². A centred 5-sample mean at any of the five positions whose window contains the glitch gives

$$\bar{a}_z = \frac{4 \times 10.35 + 50.33}{5} = 18.35 \text{ m/s}^2, \qquad n = \frac{18.35}{9.807} = 1.87.$$

Averaging did not remove the glitch; it spread it over five samples and reduced it to a plausible-looking 1.87 g, larger than the real 1.80 g pull-up. A median of 7 samples is not moved by one bad value, so despiking first leaves the pull-up as the true peak.

2. Abstract base classes: the contract of a stage

A *stage* takes data and returns new data. An **abstract base class** (ABC) states this contract in code: a class that inherits from `abc.ABC` and marks `transform` with `@abstractmethod` cannot be instantiated, and neither can any subclass that forgets to implement it. Python raises `TypeError: Can't instantiate abstract class ... with abstract method transform` at *construction*, which is far better than an `AttributeError` in the middle of a long run. An ABC may also declare **abstract properties** (`@property` on top of `@abstractmethod`), forcing each subclass to expose a value, and it may contain **concrete** methods shared by all subclasses. In the course library the concrete `PipelineStage.__call__` checks the input type, refuses to run an unfitted stage, calls `transform`, and checks the output type, so every stage gets these guarantees without writing them again.

3. Dataclasses: the configuration is the class

A stage's settings (columns, window length, threshold) *are* its identity: two stages with the same settings are the same step of a recipe. `@dataclass` writes `__init__`, `__repr__` and `__eq__` from the annotated class attributes, so equal settings mean equal objects and the printout shows every setting. Four features matter:

- **`field(default_factory=list)`.** A mutable default such as `tags: list = []` is refused with `ValueError` when the class is defined, because one list would be shared by every object (the Week 2 shared-list bug). The factory creates a new list per object.
- **`frozen=True`.** Assigning to a field raises `FrozenInstanceError`, and the object becomes hashable. A recipe that produced a published number must not change afterwards.
- **`__post_init__`.** Runs after the generated `__init__`; we use it to validate (a centred window must be odd and at least 3) and to normalise (a list of columns becomes a tuple, so the object stays hashable and a JSON list round-trips). In a frozen class the one permitted write is `object.__setattr__(self, "columns", tuple(columns))`.
- **`field(init=False, repr=False, compare=False)`.** Declares *learned state*: not a constructor argument, not printed, not compared. `Standardize` stores its learned `mean_` and `std_` this way, so a fitted and an unfitted `Standardize(["x"])` compare equal: same recipe, different stage of life. The trailing underscore is the scikit-learn convention we will meet in Week 6.

4. Writing `__repr__`, `__eq__` and `__hash__` yourself

A container such as the pipeline needs its own special methods. `__repr__` is for developers and should be unambiguous, ideally valid Python so that `eval(repr(x)) == x`; `__str__` is the short, readable form that `print` prefers. `__eq__` should return `NotImplemented` (not `False`) for an unrelated type, so Python can ask the other operand. Defining `__eq__` silently sets `__hash__ = None`; if objects are meant to live in sets or as dictionary keys, `__hash__` must hash *exactly* the data that `__eq__` compares, and that data must never change. `DataPipeline` compares its stage tuples and is deliberately unhashable, because a pipeline may contain a mutable, fittable stage.

5. Type hints, generics and run-time checks

Type hints document intent and let a checker such as `mypy` find mistakes before running, but Python does not enforce them: `peak_g("9.8")` with a hint of `np.ndarray` fails only inside the body.

A `TypeVar` is a placeholder type, and `Generic[InT, OutT]` makes a class accept type parameters, so `PipelineStage[pd.DataFrame, np.ndarray]` reads exactly like `list[int]`. These parameters are **erased** at run time, so each stage also declares `INPUT_TYPE` and `OUTPUT_TYPE` as `ClassVar` attributes. The pipeline constructor checks every neighbour pair with `issubclass(left.OUTPUT_TYPE, right.INPUT_TYPE)`; `ToArray(...) >> RollingMean(...)` is refused with a `TypeError` that names both stages, before any data flows.

6. The Hampel filter

For sample x_k and a centred window of w samples, let m_k be the window median and $\mathrm{MAD}_k = \mathrm{median}_j\,|x_j - m_k|$ the median absolute deviation. For Gaussian noise with standard deviation σ, $|x-\mu|$ has median $\Phi^{-1}(3/4)\,\sigma = 0.6745\,\sigma$, so $1.4826\,\mathrm{MAD}$ is a consistent estimate of σ that a few outliers cannot inflate. The filter replaces x_k by m_k when

$$|x_k - m_k| > n_\sigma \cdot 1.4826 \cdot \mathrm{MAD}_k\,.$$

Worked Example 2: the Hampel filter by hand

Altitudes (m): 100.0, 100.4, 99.8, **106.0**, 100.3, 100.1, 99.9; $w = 5$, $n_\sigma = 3$. At $k = 3$ the window {100.4, 99.8, 106.0, 100.3, 100.1} sorts to 99.8, 100.1, **100.3**, 100.4, 106.0, so $m_3 = 100.3$. The deviations 0.1, 0.5, 5.7, 0.0, 0.2 sort to 0.0, 0.1, **0.2**, 0.5, 5.7, so $\mathrm{MAD}_3 = 0.2$ and the threshold is $3 \times 1.4826 \times 0.2 = 0.89$ m. Since $|106.0 - 100.3| = 5.7 > 0.89$, the sample is replaced by 100.3. At $k = 4$ the median is 100.1 and $|100.3 - 100.1| = 0.2 < 0.89$: kept. Two cautions: a spike of L consecutive samples is removed only if $w \geq 2L + 1$ (the good samples must be the majority), and with a short window on pure noise the MAD itself is noisy, so some good samples are also replaced by their local median (about 4.8 % of the accelerometer samples with $w = 7$ on the flight log), a mild smoothing rather than an error.

Worked Example 3: the order of stages matters

Take $x = [1, 1, 1, 10, 1, 1, 1]$, a Hampel filter H with $w = 3$ and a centred 3-sample mean S. **Despike first:** at $k = 3$ the window {1, 10, 1} has median 1 and MAD 0, so the threshold is 0 and 10 is replaced by 1; $S(H(x))$ is all ones, peak 1. **Smooth first:** $S(x) = [1, 1, 4, 4, 4, 1, 1]$ because $(1 + 1 + 10)/3 = 4$; every window around the bump now has median 4, nothing is flagged, and $H(S(x))$ peaks at 4. Composition is not commutative, $H \circ S \neq S \circ H$, which is why a pipeline must store its order and why `H >> S` and `S >> H` compare unequal.

7. The course library: `aerospace_ai.pipeline`

`src/aerospace_ai/pipeline.py` combines the pieces. `PipelineStage(ABC, Generic[InT, OutT])` provides `transform` (abstract), `fit` (a no-op unless the stage learns), `is_fitted`, `name`, `get_params` (the dataclass fields with `init=True`), `__call__` and `>>`; its `__init_subclass__` hook registers every subclass by name when the class statement runs. The frozen dataclass stages are `SensorToFrame` (any `BaseSensor` to a DataFrame), `SelectColumns`, `DropNonFinite`, `FillGaps`, `HampelFilter`, `RollingMean`, `TimeDerivative` and `ToArray`; `Standardize` learns. `DataPipeline` stores an immutable tuple of stages, checks the chain in `__init__`, implements `__len__`, `__iter__`, `__getitem__` (a slice returns a sub-pipeline), `__repr__`, `__str__` and `__eq__`, fits each stage on the output of the one before it, and `run` returns the output together with one

`StageReport` per stage (rows in and out, NaN count, cells changed, time). `to_config` produces a plain dictionary for `json.dumps`; `from_config` looks the stage names up in the registry and rebuilds an equal pipeline. Written as pipelines, analyst A is `RollingMean(5) >> SelectColumns(...)` and analyst B is `HampelFilter(7) >> RollingMean(5) >> SelectColumns(...)`: they print differently, compare unequal, and B's saved JSON rebuilds B exactly.

The pipeline is written against `BaseSensor`, not against particular sensors, so it does not depend on the LiDAR and GPS classes of Assignment 1; students' own sensors plug into `SensorToFrame` unchanged, and the reference versions join the library after grading.

8. Learned state and leakage

Worked Example 4: fit on training data only

Training altitudes {2, 4, 6} m give $\mu = 4$ and $\sigma = \sqrt{(4 + 0 + 4)/3} = 1.633$ m, so a test sample $x = 8$ m standardises to $z = (8 - 4)/1.633 = 2.449$: well outside the training range, which is exactly what a model should be told. Fitting on training **and** test data, {2, 4, 6, 8}, gives $\mu = 5$, $\sigma = \sqrt{(9 + 1 + 1 + 9)/4} = 2.236$ m and $z = 1.342$: the test sample now looks ordinary because it helped define "ordinary". This is data leakage, the main topic of Week 5. In code, `fit` belongs to the training data only, and `DataPipeline.fit` fits each stage on what the previous, already-fitted stages produce.

9. A window of rows is not a window of seconds

A climb-rate pipeline that smooths barometric altitude over 51 *rows* and differentiates reports a maximum of 20.99 m/s at t = 99.76 s, right before a 1.2 s logger gap: near the gap the 51-row window covers more time on one side than on the other, and the altitude change across the gap leaks into the slope. Interpolating onto a uniform 10 Hz grid first removes the artefact (maximum 19.91 m/s at t = 84.0 s). Practice Assignment 3 builds that `Resample` stage, together with a learning `MinMaxScale` stage, and saves the whole air-data pipeline as JSON.

What we now know how to do

Write contracts as abstract base classes and configurations as frozen dataclasses; separate learned state with `field(init=False, repr=False, compare=False)`; write `__repr__`, `__eq__` and a consistent `__hash__`; use `TypeVar` and `Generic` and check types where Python does not; despike with a Hampel filter by hand and in code; and compose typed stages into a pipeline that reports what each stage did and saves itself for reproducibility. Next week: the Strategy, Observer and Factory patterns, and pytest in depth.